# INSTITUCIÓN UNIVERSITARIA PASCUAL BRAVO

Tecnología en Desarrollo de Software
Bases de Datos I — Grupo G100

# TAREA 3

Implementación del modelo físico de la Red Social Estudiantil Pascualina

Estudiante: Alexandra Zapata
Docente: Oralia Cortes Grajales
Fecha: 18/10/2026

## Introducción

Este informe presenta la implementación física del modelo lógico de la Red Social Estudiantil Pascualina en PostgreSQL. La plataforma busca facilitar la interacción académica mediante perfiles, publicaciones, comentarios, reacciones, grupos, eventos, intereses, habilidades, seguimiento y mentoría. La implementación concreta el modelo relacional mediante tablas, tipos de datos, claves primarias, claves foráneas y restricciones de integridad.

La implementación se basa en el documento lógico de la Tarea 2, compuesto por quince tablas. El script y el informe deben revisarse en conjunto para confirmar que nombres, atributos y relaciones coincidan.

## 1. Identificación y justificación del SGBD

Se selecciona PostgreSQL por ser un sistema gestor relacional de código abierto que ofrece transacciones, integridad referencial, restricciones declarativas y herramientas gráficas como pgAdmin 4. Estas capacidades son adecuadas para una base de datos con relaciones entre perfiles, publicaciones, grupos, eventos y mentorías.

MySQL es una alternativa madura y extendida, mientras que MariaDB mantiene compatibilidad con muchas aplicaciones del ecosistema MySQL. PostgreSQL se elige para esta actividad por sus capacidades relacionales y su integración con pgAdmin 4. Como desventaja práctica, requiere familiarizarse con su entorno y particularidades; esto no significa que sea la mejor opción para todos los escenarios.

## 2. Transformación del modelo lógico al modelo físico

Las entidades se implementan como tablas y sus atributos como columnas. Las entidades principales utilizan claves primarias simples; las relaciones N:M se resuelven mediante tablas intermedias con claves primarias compuestas. PUBLICACION no tiene una FK directa a ESTUDIANTE, ya que la autoría compartida se representa en PUBLICACION_ESTUDIANTE. SEGUIR y MENTORIA son relaciones reflexivas con dos roles de estudiante.

Se usan INTEGER IDENTITY para identificadores artificiales, VARCHAR con límites definidos para textos, SMALLINT para semestre, DATE para fechas sin hora y TIMESTAMP cuando se requiere registrar el momento exacto. Las restricciones CHECK validan valores categóricos y evitan el autoseguimiento y la automentoría.

## 3. Inventario de tablas

| Tabla | Propósito |
|---|---|
| **ESTUDIANTE** | Perfil y datos académicos. |
| **HABILIDAD** | Catálogo de habilidades. |
| **INTERES** | Catálogo de intereses. |
| **ESTUDIANTE_HABILIDAD** | Relación N:M entre estudiantes y habilidades. |
| **ESTUDIANTE_INTERES** | Relación N:M entre estudiantes e intereses. |
| **GRUPO** | Grupos de estudio, clubes y equipos. |
| **ESTUDIANTE_GRUPO** | Membresía y rol en grupos. |
| **EVENTO** | Información de eventos. |
| **ASISTENTE_EVENTO** | Relación entre estudiantes y eventos; registra estado/rol. |
| **PUBLICACION** | Contenido del feed. |
| **PUBLICACION_ESTUDIANTE** | Autoría N:M de publicaciones. |
| **COMENTARIO** | Comentarios sobre publicaciones. |
| **REACCION** | Reacción de un estudiante a una publicación. |
| **SEGUIR** | Seguimiento reflexivo entre estudiantes. |
| **MENTORIA** | Mentoría entre estudiantes asociada a una habilidad. |

## 4. Descripción técnica de las tablas

### ESTUDIANTE

id_estudiante INTEGER PK IDENTITY; nombre VARCHAR(100) NOT NULL; email VARCHAR(150) NOT NULL UNIQUE; contrasena VARCHAR(255) NOT NULL; programa VARCHAR(100) NOT NULL; area_estudio VARCHAR(100); semestre SMALLINT; ciudad VARCHAR(100); biografia VARCHAR(500); foto_url VARCHAR(255); fecha_registro DATE; fecha_actualizacion TIMESTAMP. Email único evita cuentas duplicadas; contrasena debe contener un hash, no texto plano.

### HABILIDAD

id_habilidad INTEGER PK IDENTITY; nombre VARCHAR(100) NOT NULL; tipo VARCHAR(50) NOT NULL; UNIQUE(nombre,tipo). La combinación evita repetir la misma habilidad dentro de una categoría.

### INTERES

id_interes INTEGER PK IDENTITY; nombre VARCHAR(100) NOT NULL UNIQUE. El identificador artificial no depende del texto del interés.

### ESTUDIANTE_HABILIDAD

id_estudiante INTEGER PK/FK; id_habilidad INTEGER PK/FK; nivel VARCHAR(30) NOT NULL; fecha_agregado DATE NOT NULL. La PK compuesta impide repetir una habilidad para el mismo estudiante.

### ESTUDIANTE_INTERES

id_estudiante INTEGER PK/FK; id_interes INTEGER PK/FK; fecha_agregado DATE NOT NULL. La PK compuesta evita duplicar la misma asociación.

### GRUPO

id_grupo INTEGER PK IDENTITY; nombre VARCHAR(100) NOT NULL; descripcion VARCHAR(500); tipo VARCHAR(50) NOT NULL; fecha_creacion DATE NOT NULL.

### ESTUDIANTE_GRUPO

id_estudiante INTEGER PK/FK; id_grupo INTEGER PK/FK; rol VARCHAR(30) NOT NULL; fecha_union DATE NOT NULL. La PK compuesta identifica una membresía por estudiante y grupo.

### EVENTO

id_evento INTEGER PK IDENTITY; titulo VARCHAR(150) NOT NULL; descripcion VARCHAR(500); tipo VARCHAR(50) NOT NULL; fecha_hora TIMESTAMP NOT NULL; lugar VARCHAR(150). TIMESTAMP conserva la hora del evento.

### ASISTENTE_EVENTO

id_estudiante INTEGER PK/FK; id_evento INTEGER PK/FK; estado VARCHAR(30) NOT NULL; fecha_registro DATE NOT NULL. La PK compuesta impide duplicar la participación del estudiante en el mismo evento.

### PUBLICACION

id_publicacion INTEGER PK IDENTITY; contenido VARCHAR(1000) NOT NULL; tipo VARCHAR(30) NOT NULL; fecha_creacion TIMESTAMP NOT NULL. No tiene FK directa a estudiante porque la autoría N:M está en PUBLICACION_ESTUDIANTE.

### PUBLICACION_ESTUDIANTE

id_publicacion INTEGER PK/FK; id_estudiante INTEGER PK/FK; rol_autor VARCHAR(30) NOT NULL; fecha_autoria TIMESTAMP NOT NULL. La PK compuesta permite varios autores sin duplicar el vínculo.

### COMENTARIO

id_comentario INTEGER PK IDENTITY; id_publicacion INTEGER FK NOT NULL; id_estudiante INTEGER FK NOT NULL; texto VARCHAR(500) NOT NULL; fecha_comentario TIMESTAMP NOT NULL.

### REACCION

id_publicacion INTEGER PK/FK; id_estudiante INTEGER PK/FK; tipo_reaccion VARCHAR(20) NOT NULL; fecha_reaccion TIMESTAMP NOT NULL. La PK compuesta limita a una reacción por estudiante y publicación.

### SEGUIR

id_seguidor INTEGER PK/FK; id_seguido INTEGER PK/FK; fecha_seguimiento DATE NOT NULL. La PK compuesta evita duplicar el seguimiento y CHECK evita autoseguimiento.

### MENTORIA

id_mentor INTEGER PK/FK; id_aprendiz INTEGER PK/FK; id_habilidad INTEGER PK/FK; estado VARCHAR(30) NOT NULL; fecha_inicio DATE NOT NULL. La clave de tres columnas distingue mentorías por habilidad.

## 5. Relaciones y acciones referenciales

Las relaciones 1:N se implementan con claves foráneas en la tabla hija: PUBLICACION–COMENTARIO y ESTUDIANTE–COMENTARIO. Las relaciones N:M usan ESTUDIANTE_HABILIDAD, ESTUDIANTE_INTERES, ESTUDIANTE_GRUPO, ASISTENTE_EVENTO, PUBLICACION_ESTUDIANTE y REACCION. SEGUIR y MENTORIA son relaciones reflexivas.

ON DELETE CASCADE se utiliza para asociaciones y datos dependientes, evitando filas huérfanas cuando se elimina uno de los registros relacionados. En MENTORIA, la FK a HABILIDAD utiliza ON DELETE RESTRICT para impedir eliminar una habilidad que todavía se utiliza. ON UPDATE CASCADE propaga cambios de identificadores referenciados, aunque en una aplicación normal conviene mantener estables las claves.

## 6. Explicación del script DDL

El archivo modelo_fisico.sql abre una transacción, crea primero las tablas base y luego las tablas intermedias y dependientes, define PK, FK, UNIQUE y CHECK, agrega índices complementarios y finaliza con COMMIT. El orden de creación respeta las dependencias entre claves foráneas.

## 7. Evidencia del modelo físico

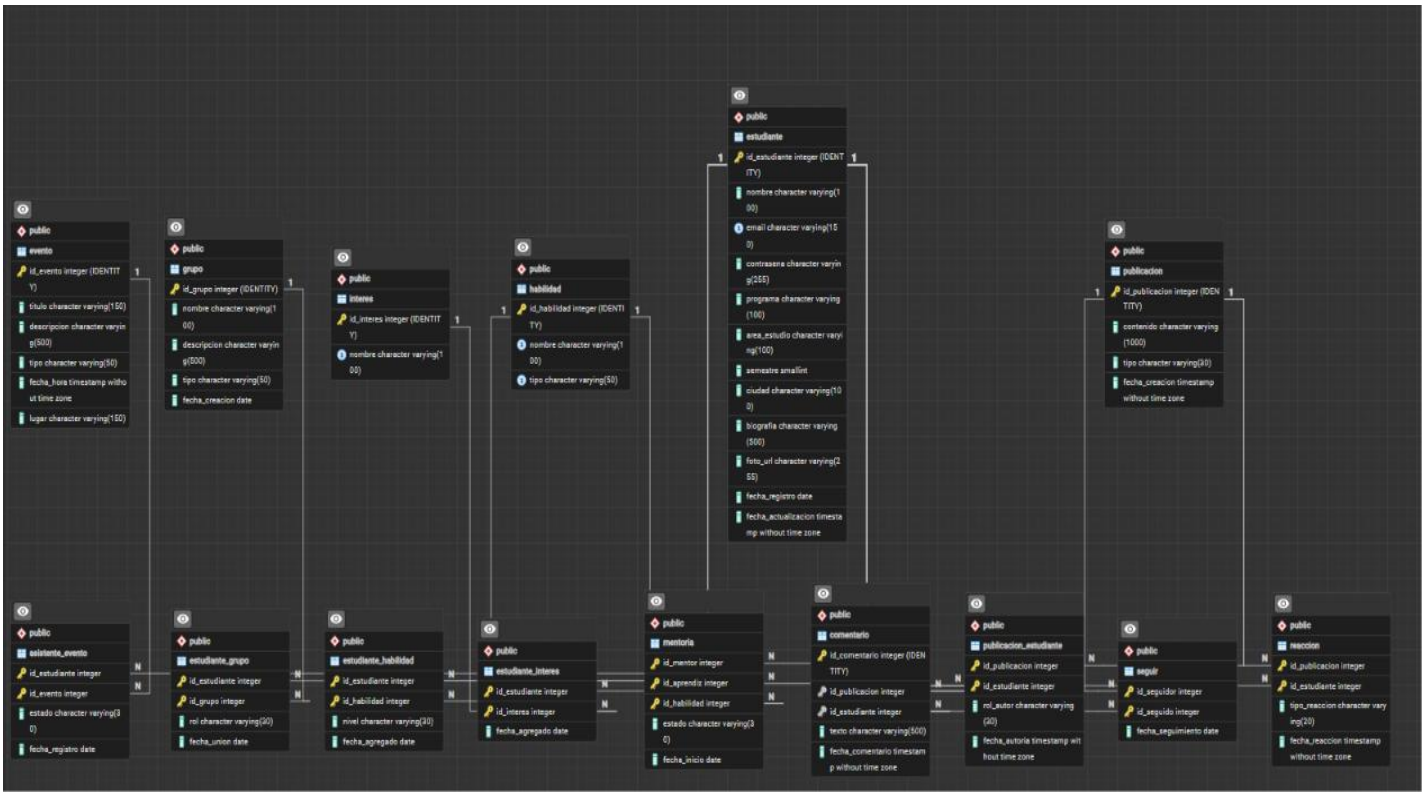



*Figura 1. Diagrama ER generado en pgAdmin 4 a partir del script DDL.*

## 8. Conclusiones individual

Al desarrollar esta tarea, el paso del plano teórico (el modelo lógico) a la ejecución real en PostgreSQL fue un reto muy enriquecedor. Me di cuenta de que un script DDL no es solo texto, sino el conjunto de reglas estrictas que van a regir la base de datos. Pasar de dibujar tablas en un papel a escribir el código en pgAdmin me permitió entender que la precisión es fundamental: un punto y coma mal ubicado o una instrucción fuera de contexto pueden detener todo el proceso.

Uno de los momentos que más me marcó fue cuando, al intentar ejecutar el script por primera vez, la consola me arrojó un error de sintaxis en la palabra CONSTRAINT. Aunque al principio frustró un poco, me enseñó una valiosa lección: que el motor de la base de datos ejecuta exactamente lo que se le indica. A partir de ese error, aprendí a revisar la estructura completa de mis tablas antes de presionar "Play" y a comprender la importancia de respetar el orden de las claves foráneas.

Asimismo, comprendí a fondo la función de las llaves primarias y las acciones referenciales. Ver en el diagrama generado en pgAdmin cómo se conectaban las 15 tablas me hizo entender que el ON DELETE CASCADE es indispensable para mantener la coherencia en tablas dependientes (como los comentarios o reacciones), mientras que usar RESTRICT en la mentoría evita perder información histórica valiosa. Las restricciones CHECK fueron otro gran aprendizaje, pues me demostraron que la base de datos puede protegerse a sí misma de datos inválidos (como un semestre fuera de rango), sin necesidad de depender únicamente de lo que haga la aplicación web.

En definitiva, esta tarea me dejó una base sólida sobre cómo estructurar un sistema de información real. Ya no veo las bases de datos como un simple dibujo estático, sino como un sistema activo donde cada restricción tiene un propósito definido. Me siento preparada para el siguiente paso de la asignatura: manipular la información y realizar consultas sobre este modelo que acabo de construir.

## Referencias

Canepa, G. (2016). PostgreSQL Database Handbook. http://dev.cs.ovgu.de/db/PostgreSQL-Database-Handbook.pdf

Elmasri, R., & Navathe, S. A. (2007). Algoritmos de diseño de bases de datos relacionales y dependencias adicionales. En Fundamentos de sistemas de bases de datos (5.ª ed., pp. 317–344). Pearson Education.

Hsu, L., & Obe, R. (2015). Essential PostgreSQL. DZone RefCardz. https://ugoproto.github.io/ugodoc/pdf/52492-rc071_postgresql_2.pdf

Mannino, M. V. (2007). Administración de bases de datos: diseño y desarrollo de aplicaciones (3.ª ed.). McGraw-Hill Interamericana.

PostgreSQL Global Development Group. (s. f.). PostgreSQL documentation. https://www.postgresql.org/docs/